

Estrategia de Recuperación de Base de Datos — RDAM

Registro de Deudores Alimentarios Morosos
Alumno: Serovich Emilio — DNI 43.770.166 — 3° T.S "A"

IZT Software Factory | Campus de Verano 2026

1. Objetivos de Recuperación

PARÁMETRO	VALOR	DESCRIPCIÓN
RPO (Recovery Point Objective)	1 hora	Máxima pérdida de datos aceptable
RTO (Recovery Time Objective)	4 horas	Tiempo máximo para restaurar el servicio
Disponibilidad objetivo	99.5%	En horario laboral L-V 8-18 hs

2. Tipos de Backup

Backup Completo (Full)

Herramienta: pg_dump / pg_basebackup
Frecuencia: Diaria — 02:00 hs
Retención: 30 días
Formato: Directorio comprimido (-Fd -Z9)

Incremental (WAL Archiving)

Herramienta: WAL-G o pgBackRest
Frecuencia: Continuo — cada 5 min
Retención: 7 días de WALs
Destino: s3://rdam-backups/wal/

Lógico (por tabla)

Herramienta: pg_dump --table
Frecuencia: Semanal — domingos 03:00
Tablas: solicitudes, historial_estados, certificados
Formato: SQL comprimido (.sql.gz)

Comando de Backup Completo

```
# Backup completo con pg_dump
pg_dump \
  --host=localhost \
  --port=5432 \
  --username=rdam_backup \
  --dbname=rdam_db \
  --format=directory \
  --compress=9 \
  --file=/backups/full/rdam_$(date +%Y%m%d_%H%M%S)

# Subir a S3
aws s3 sync /backups/full/ s3://rdam-backups/full/ --delete
```

3. Esquema de Retención

Backups diarios completos → 30 días
WAL archivados → 7 días
Backups semanales lógicos → 12 semanas (3 meses)
Backups mensuales → 12 meses

4. Procedimiento de Restauración

4.1 Restauración Completa (Full Restore)

⚠ Usar cuando la base de datos es irrecuperable o hay corrupción total.

```
# 1. Detener la aplicación
systemctl stop rdam-api

# 2. Descargar el último backup completo
aws s3 cp s3://rdam-backups/full/rdam_YYYYMMDD_HHMMSS /tmp/restore/ --recursive

# 3. Eliminar la base de datos corrupta
psql -U postgres -c "DROP DATABASE IF EXISTS rdam_db;"
psql -U postgres -c "CREATE DATABASE rdam_db OWNER rdam_user;"

# 4. Restaurar
pg_restore \
  --host=localhost \
  --username=postgres \
  --dbname=rdam_db \
  --format=directory \
  --jobs=4 \
  /tmp/restore/rdam_YYYYMMDD_HHMMSS

# 5. Verificar integridad
psql -U postgres -d rdam_db -c "SELECT COUNT(*) FROM solicitudes;"
psql -U postgres -d rdam_db -c "SELECT COUNT(*) FROM usuarios;"

# 6. Reiniciar la aplicación
systemctl start rdam-api
```

4.2 Restauración a Punto en el Tiempo (PITR)

⚠ Usar cuando se necesita recuperar hasta un momento específico (ej: antes de un error humano).
Nota: A partir de PostgreSQL 12, recovery.conf está deprecado. Se usa recovery.signal + parámetros en postgresql.auto.conf .

```
# 1. Detener PostgreSQL
systemctl stop postgresql

# 2. Restaurar base de backup con WAL-G
wal-g backup-fetch /var/lib/postgresql/data LATEST

# 3. Crear archivo signal y configurar recovery (PostgreSQL 12+)
touch /var/lib/postgresql/data/recovery.signal

cat >> /var/lib/postgresql/data/postgresql.auto.conf << EOF
restore_command = 'wal-g wal-fetch %f %p'
recovery_target_time = '2026-02-10 14:30:00'
recovery_target_action = 'promote'
EOF

# 4. Iniciar PostgreSQL (modo recovery)
systemctl start postgresql
# PostgreSQL detecta recovery.signal y entra en modo PITR.
# Al alcanzar el target, promueve automáticamente.

# 5. Monitorear recuperación
tail -f /var/log/postgresql/postgresql.log | grep -E "recovery|PITR"

# 6. Verificar promoción exitosa
ls /var/lib/postgresql/data/recovery.signal 2>/dev/null \
  && echo "AÚN EN RECOVERY" || echo "PROMOVIDO OK"
```

4.3 Restauración de Tabla Específica

⚠ Usar cuando se borra o corrompe una tabla puntual.

```
# Restaurar solo la tabla solicitudes desde backup semanal
pg_restore \
  --host=localhost \
  --username=postgres \
  --dbname=rdam_db \
  --table=solicitudes \
  /backups/weekly/rdam_weekly_YYYYMMDD.dump
```

5. Verificación de Backups

Los backups deben verificarse automáticamente. Ejecutar semanalmente:

```
#!/bin/bash - verify-backup.sh

BACKUP_FILE=$(ls -t /backups/full/ | head -1)
TEST_DB="rdam_verify_$(date +%s)"

echo "Verificando backup: $BACKUP_FILE"

# Restaurar en DB temporal
psql -U postgres -c "CREATE DATABASE $TEST_DB;"
pg_restore --dbname=$TEST_DB --format=directory /backups/full/$BACKUP_FILE

# Verificaciones mínimas
USERS=$(psql -U postgres -d $TEST_DB -tAc "SELECT COUNT(*) FROM usuarios;")
SOLS=$(psql -U postgres -d $TEST_DB -tAc "SELECT COUNT(*) FROM solicitudes;")

echo "Usuarios: $USERS | Solicitudes: $SOLS"

if [ "$USERS" -gt 0 ] && [ "$SOLS" -ge 0 ]; then
  echo "✓ Backup verificado correctamente"
else
  echo "✗ ERROR: Backup inválido - alertar al equipo"
fi

# Limpiar
psql -U postgres -c "DROP DATABASE $TEST_DB;"
```

6. Monitoreo y Alertas

EVENTO	CANAL DE ALERTA	SLA DE RESPUESTA
Backup fallido	Email + Slack	30 minutos
Espacio en disco > 80%	Email	2 horas
Retraso WAL > 10 min	PagerDuty	15 minutos
Conexiones > 90% del límite	Slack	1 hora
Tiempo de query > 5s	Log + Slack	4 horas

Queries de Monitoreo

```
-- Tamaño de la base de datos
SELECT
  pg_database.datname,
  pg_size_pretty(pg_database_size(pg_database.datname)) AS size
FROM pg_database
WHERE datname = 'rdam_db';

-- Conexiones activas
SELECT count(*), state
FROM pg_stat_activity
WHERE datname = 'rdam_db'
GROUP BY state;
```

7. Runbook de Desastre (Disaster Recovery)

Escenario A — Corrupción de datos por error humano

- Identificar el timestamp del incidente en los logs
- Ejecutar PITR al momento anterior al incidente (sección 4.2)
- Validar integridad con queries de verificación
- Reiniciar aplicación y notificar a usuarios afectados

Escenario B — Falla total del servidor de base de datos

- Provisionar nuevo servidor PostgreSQL (misma versión)
- Ejecutar restauración completa desde S3 (sección 4.1)
- Aplicar WALs desde el último backup hasta el punto de falla
- Actualizar variables de entorno de la API con la nueva IP/host
- Reiniciar la API y verificar conectividad

Escenario C — Eliminación accidental de tabla

- Identificar la tabla afectada y el timestamp
- Ejecutar restauración de tabla específica (sección 4.3)
- Verificar integridad referencial con FKs
- Documentar el incidente en el registro de cambios

8. Roles y Responsabilidades

ROL	RESPONSABILIDAD
DBA/DevOps	Ejecutar backups, monitorear, responder alertas
Tech Lead	Aprobar restauraciones en producción
Admin RDAM	Notificar a usuarios durante ventana de mantenimiento

9. Checklist de Recuperación

- ☐ Identificar tipo y alcance del incidente
- ☐ Notificar al Tech Lead y Admin
- ☐ Seleccionar estrategia de recuperación (Full / PITR / Tabla)
- ☐ Ejecutar restauración en ambiente de prueba primero
- ☐ Validar integridad de datos restaurados
- ☐ Ejecutar restauración en producción
- ☐ Verificar funcionamiento de la aplicación
- ☐ Documentar el incidente (causa, impacto, acciones, duración)
- ☐ Revisar y mejorar controles para evitar recurrencia

10. Expiración Automática de Certificados

El DDL incluye la función fn_expirar_certificados() que marca como EXPIRADA las solicitudes con certificados vencidos. Debe programarse con pg_cron :

```
-- Instalar extensión (una vez)
CREATE EXTENSION IF NOT EXISTS pg_cron;

-- Programar ejecución diaria a las 03:00 hs
SELECT cron.schedule(
  'expire-certs',
  '0 3 * * *',
  'SELECT fn_expirar_certificados()'
);

-- Verificar jobs activos
SELECT * FROM cron.job;
```

Alternativamente, puede invocarse desde un cron del sistema operativo:

```
# crontab -e
0 3 * * * psql -U rdam_user -d rdam_db -c "SELECT fn_expirar_certificados();"

```